

The AWS ChatOps Agent

Operate your cloud from a chat box. Ask in plain English, let a gpt-4o agent choose the right AWS tools — and approve every change with one click.

gpt-4o

AGENT BRAIN

boto3

AWS TOOLS

Streamlit

LIVE UI

Prepared by Aravind Baranitharan · AI Agents for Cloud & DevOps

Why talk to your cloud in **plain English**?

The daily pain

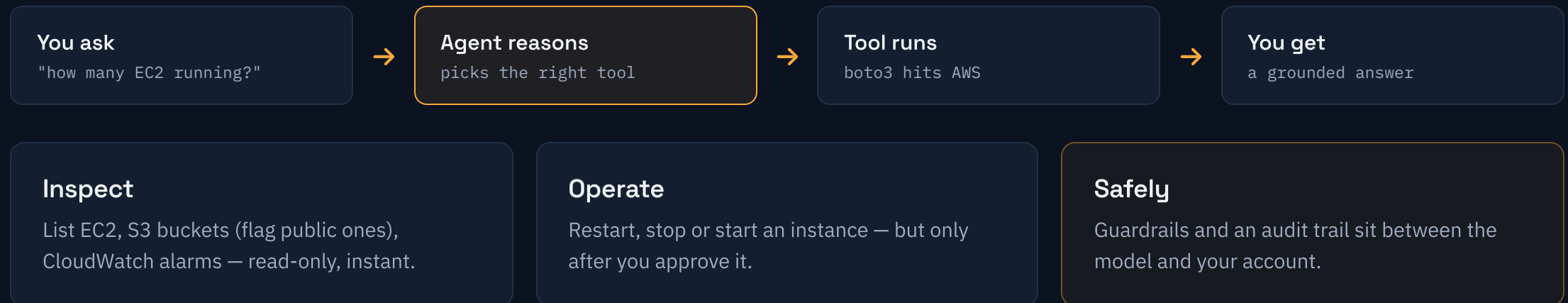
- Ops is constant **context-switching** — console tabs, CLI flags, dashboards.
- Answers live in people's heads: "which box is the staging worker?"
- Manual changes are risky — one wrong **stop-instances** hits prod.

The ChatOps answer

- One chat box to **ask** and **act**, in plain language.
- The agent knows the tools — it looks things up and does the wiring.
- Every state change is **proposed first** and needs a human OK.

In one line: a conversational SRE that reads your AWS account and takes safe, approved actions — so routine ops stop needing a console expert.

An agent that reasons, then acts



Running live on a **real AWS** account

Connected to account **836471808669** through a least-privilege IAM user. Actual results from the demo:

- **EC2 · us-east-1** → **BotxStudio** (t3.small, running)
- **EC2 · ap-south-1** → **markbot-production** (t3.medium)
- **S3** → 13 buckets; flagged **ombak-product-images** as ⚠ public
- **Action** → "restart BotxStudio" → approval gate → done

chatops – live

you ▶ "how many running EC2 in us-east-1?"

gpt-4o chooses a tool

agent ▶ `list_ec2_instances(`
 `region="us-east-1",`
 `state="running")`

boto3 answers from your account

tool ◀ 1 instance: **BotxStudio** running

Same code runs offline on mock data, or live on real AWS.

The key idea: tool calling

The LLM never touches AWS. It only **asks** for a tool by name, with arguments. **Your code** runs that tool with **boto3** and hands the result back. That separation is what makes an agent both powerful and safe.

- A tool = a **name** + a **description** + a typed **schema**.
- The model reads the description to decide **when** to call it.
- You stay in control of **what actually executes**.

The round-trip

```
model: call restart_ec2_instance(i-03b...)
```

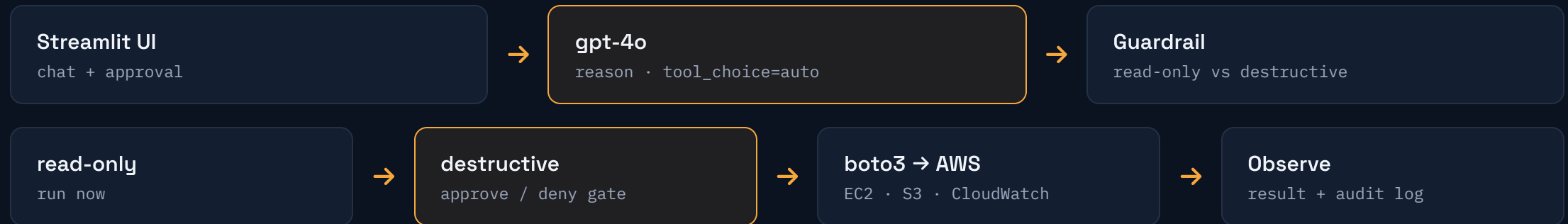
```
your code executes it (boto3)
```

```
result returned to the model
```

```
model explains what happened
```

No tool means no action. The model can only do what you gave it — and destructive tools wait for you.

End-to-end data flow



Bounded loop: the model may chain tools — look up an id, then act — but the loop is capped at 8 steps and no state-changing call runs without an explicit approval.

Perceive, reason, **act** — repeat

Each turn, the whole conversation — system, history, tool results — is re-sent to gpt-4o. If it returns **tool_calls**, we run them and loop; if it returns text, we're done. One loop drives both the live model and the offline fallback.

- **temperature = 0** → deterministic, demo-safe.
- Tool errors return as **observations**, not crashes.
- A hard **step cap** means the loop can never run away.

agent.py · the loop

```
resp = client.chat.completions.create(
    model="gpt-4o", messages=messages,
    tools=TOOLS, tool_choice="auto",
    temperature=0)

msg = resp.choices[0].message
# did the model ask for tools?
for tc in msg.tool_calls:
    out = execute_tool(tc.name, tc.args)
    messages.append(tool_result(tc.id, out))
```

agent.py · call_model() — the core round-trip

The agent proposes. **A human approves.**

Every tool carries a **risk label**. Read-only tools run automatically; anything that changes state is **held** and surfaced as an Approve / Deny button. Nothing mutates your account silently.

- `read-only` list · describe · inspect → run now
- `destructive` restart · stop · start → wait for approval
- **Audit log** — every action, decision & timestamp recorded.

aws_tools.py · RISK

```
RISK = {  
    "list_ec2_instances": "read_only",  
    "get_cloudwatch_alarms": "read_only",  
    "restart_ec2_instance": "destructive",  
    "stop_ec2_instance": "destructive",  
}  
  
# destructive -> pause the loop for approval  
if RISK[name] == "destructive":  
    st.session_state.pending = call
```

One map drives the entire safety gate.

Services, and two ways to run it

EC2

List by region/state; restart · stop · start (gated).

S3

List buckets and flag any that are publicly accessible.

CloudWatch

List alarms, optionally filtered by state.

Mock mode

`USE_REAL_AWS=false` — deterministic fake inventory. Zero risk, no account needed. Ideal for teaching and on-stage.

Real mode

`USE_REAL_AWS=true` — boto3 against your account via the standard credential chain. Same code, real resources.

Six files, clear jobs

FILE	RESPONSIBILITY
aws_tools.py	The AWS tools (mock + real boto3), the OpenAI tool schemas, and the RISK map.
agent.py	The gpt-4o tool-calling brain (call_model) plus an offline fallback.
app.py	The Streamlit UI: chat, the loop driver, the Approve / Deny gate, the audit log.
test_connection.py	Read-only pre-flight — verifies credentials and each IAM permission.
iam-policy.json	Least-privilege IAM policy — exactly the actions the app uses.
env.sample · requirements.txt	Config template (rename to .env) and dependencies.

A tool = function + schema

Each tool is a plain Python function that works in **mock** or **real** mode, plus an OpenAI schema the model reads. In real mode it's a one-line boto3 call.

- **TOOLS** → the schemas passed to gpt-4o.
- **REGISTRY** → name → the Python function.
- **RISK** → name → read-only or destructive.

aws_tools.py

```
def restart_ec2_instance(instance_id, region=None):
    region = region or DEFAULT_REGION
    if _use_real():
        boto3.client("ec2", region_name=region)\
            .reboot_instances(InstanceIds=[instance_id])
    return f"Reboot requested for {instance_id}"
... # mock path for offline demos

REGISTRY = {"restart_ec2_instance": restart_ec2_instance}
```

One function, two modes – mock for demos, boto3 for real.

The brain, UI-agnostic

`call_model()` runs one turn and returns a normalized result: the final text, or the tool calls the model wants. It works with a real key (gpt-4o) and has a small offline fallback so it demos with no key.

- Live: OpenAI SDK, `tools=TOOLS, tool_choice="auto"`.
- `execute_tool()` runs a chosen tool safely.
- Config (key, model, region) loaded from `.env`.

```
agent.py

def call_model(messages):
    # -> (content, calls[{id,name,args}], assistant)
    if has_key():
        return _live(messages)    # real gpt-4o
    return _offline(messages)    # keyword fallback

def execute_tool(name, args):
    return REGISTRY[name](**args) # run the tool
```

Same interface, live or offline – the UI never changes.

Streamlit UI & the approval gate

`drive()` runs model turns, auto-executing read-only tools, until a final answer or a destructive action needs approval. Then it pauses; the UI renders the buttons, and `resolve()` finishes the action on your click.

- Chat history and tool traces render as messages.
- **Approve** → run it & log; **Deny** → skip, log the block.
- Sidebar shows mode, brain, and the live audit log.

```
app.py · drive()

for c in calls:
    if RISK[c.name] == "destructive":
        st.session_state.pending = c    # pause
    else:
        out = execute_tool(c.name, c.args)    # run now

# on the human's click:
if approve: execute_tool(c.name, c.args); audit("approved")
if deny:    audit("denied")    # nothing runs
```

The whole safety story, in a dozen lines.

Least privilege, **by design**

01

Scoped IAM

A dedicated user with only the actions the app calls.

02

No secrets in code

Keys live in `.env` — `chmod 600`, `git-ignored`.

03

Human gate

No restart/stop/start without an explicit approval.

04

Audit trail

Every action and decision, timestamped.

Read-only first: a pre-flight (`test_connection.py`) verifies each permission before you demo. Start with inspect-only tools, add actions deliberately.

A safe, conversational **cloud operator**

- ✓ Plain-English ops over EC2, S3 and CloudWatch.
- ✓ gpt-4o + tool calling + boto3 — the model reasons, your code acts.
- ✓ Guardrails: read-only runs free, changes need approval.
- ✓ Runs live on real AWS or fully offline on mock data.
- ✓ `streamlit run app.py` — that's the whole demo.

Where it goes next

From ChatOps to autonomy

Add a **FinOps** tool for idle spend, then wire alarms → the agent to build **auto-remediation**: detect, propose, approve, fix, verify.

agentic ops → self-healing